

Edukia — API Fichiers (R2) pour le mobile

Téléverser & télécharger via des URL présignées · Cloudflare R2 · exemples issus d'appels **réels** (dev)

1. Principe (à lire en premier)

Le backend ne transporte **jamais** les octets. Il délivre des **URL présignées** de courte durée ; l'app dialogue **directement** avec Cloudflare R2 (PUT pour téléverser, GET pour télécharger).

Visibilité du slot	Où se trouve l'URL	Comment la lire
public (photos, logos, icônes...)	La colonne <code><slot>_path</code> contient déjà l'URL publique complète .	On l'utilise directement. Ne PAS appeler download-url .
privé (PDF d'épreuves, pièces d'identité...)	La ligne ne stocke qu'un chemin logique.	Appeler <code>GET .../download-url</code> (GET présigné temporaire).

- Tous les endpoints `/files/*` exigent `Authorization: Bearer <JWT>`.
- Clé R2 déterministe : `<entity>/<uuid>/<slot>.<ext>`. Un nouveau téléversement écrase en place.
- URL de base (prod) : `https://api.educ-prime.com`

2. Découvrir — `GET /files/registry`

Renvoie `entity → slot → { authorized, public }`. À récupérer une fois puis mettre en cache.

```
// Extrait réel
{
  "recruteurs": { "profil": { "authorized": ["jpg","jpeg","png","webp","avif"], "public": true } },
  "epreuves": { "file": { "authorized": ["pdf"], "public": false } }
}
```

3. Téléverser (2 étapes) — public OU privé

Flux **identique** pour public ou privé ; seule diffère la liste des en-têtes exigés par R2 (dans `required_headers`).

Étape 1 — demander une URL de téléversement

POST `/files/<entity>/<uuid>/<slot>/upload-url` · corps : `{ "extension": "png" }`

```
// Réponse RÉELLE — slot PUBLIC recruteurs/profil
{
  "url": "https://<compte>.r2.cloudflarestorage.com/recruteurs/06255da6-.../profil.png?X-Amz-Credential=<REDACTED>&X-Amz-Expires=600&X-Amz-Signature=<REDACTED>&x-id=PutObject",
  "method": "PUT",
  "content_type": "image/png",
  "required_headers": { "Content-Type": "image/png", "Cache-Control": "public, max-age=31536000, immutable" },
  "path": "https://assets-dev.edukia.net/recruteurs/06255da6-.../profil.png", // URL publique
  "extension": "png", "expires_in": 600, "public": true
}
```

Slot PRIVÉ (epreuves/file) : idem **sauf** `content_type: "application/pdf"`, `required_headers: { "Content-Type": "application/pdf" }` (**pas** de `Cache-Control`), `path: "/epreuves/<uuid>/file"`, `expires_in: 3600`, `public: false`.

Étape 2 — PUT des octets directement vers R2

Critique : rejouer toutes les clés de `required_headers` à l'identique. `Content-Type` toujours ; `Cache-Control` pour les slots **publics**. Slot public sans `Cache-Control` ⇒ R2 renvoie `401 SignatureDoesNotMatch`.

```
// Flutter / Dart
final res = await http.post(Uri.parse('$base/files/recruteurs/$uuid/profil/upload-url'),
  headers: {'Authorization': 'Bearer $jwt', 'Content-Type': 'application/json'},
  body: jsonEncode({'extension': 'png'}));
final u = jsonDecode(res.body);

// PUT vers R2 – rejouer TOUS les required_headers
final put = await http.put(Uri.parse(u['url']),
  headers: Map<String,String>.from(u['required_headers']), body: fileBytes);
// put.statusCode == 200 ⇒ terminé (test réel : HTTP 200)
```

Slot public : après le PUT, `<slot>_path` (ex. `profil_photo_path`) contient l'URL publique complète — on l'affiche directement. **Aucun appel à `download-url`**.

Variante — téléversement relayé par le serveur (1 appel)

POST `/files/<entity>/<uuid>/<slot>/upload` — `multipart/form-data`, champ `file`. Le serveur pousse les octets vers R2 à votre place.

4. Télécharger depuis le stockage PRIVÉ — **GET** `.../download-url`

Uniquement pour les slots **privés**. Renvoie `400` pour un slot public (lire `<slot>_path` à la place).

```
// Réponse RÉELLE – slot PRIVÉ epreuves/file – HTTP 200
{
  "url": "https://<compte>.r2.cloudflarestorage.com/epreuves/1431ba81-.../file.pdf?X-Amz-Credential=<REDACTED>&X-Amz-Expires=3600&X-Amz-Signature=<REDACTED>&x-id=GetObject",
  "method": "GET", "path": "/epreuves/1431ba81-.../file", "extension": "pdf",
  "expires_in": 3600, "public": false
}
```

```
// Flutter / Dart
final r = await http.get(Uri.parse('$base/files/epreuves/$uuid/file/download-url'),
  headers: {'Authorization': 'Bearer $jwt'});
final url = jsonDecode(r.body)['url'];
// GET des octets depuis R2 – AUCUN en-tête Authorization (la signature EST l'auth)
final pdf = await http.get(Uri.parse(url)); // pdf.bodyBytes
```

```
// download-url sur un slot PUBLIC ⇒ 400 (message RÉEL)
{ "statusCode": 400, "error": "Bad Request",
  "message": "Slot 'recruteurs/profil' is public – no presigned download URL is issued. Read its URL straight from the entity's 'profil_photo_path' field; a plain GET on that URL serves the file." }
```

5. Règles & pièges

Règle	Détail
Auth	<code>Authorization: Bearer <JWT></code> sur tous les appels <code>/files/*</code> . L'URL R2 elle-même : aucun en-tête.
Corps upload-url	Exactement <code>{ "extension": "png" }</code> . Envoyer <code>content_type / filename</code> ⇒ <code>400 "property ... should not exist"</code> .
Rejouer en-têtes	Le PUT inclut chaque <code>required_headers</code> . Public sans <code>Cache-Control</code> ⇒ <code>401 SignatureDoesNotMatch</code> .
Lecture publique	Lire <code><slot>_path</code> de l'entité. <code>download-url</code> ⇒ <code>400</code> pour public.
Lecture privée	Toujours via <code>download-url</code> . Jamais d'URL R2 codée en dur.
Extensions	Doit figurer dans <code>authorized</code> (voir <code>/files/registry</code>). Sinon <code>400</code> .
Expiration	Téléversement ~10 min ; téléchargement 10 min (1 h pour PDF épreuves/concours). Re-demander si expirée.

6. Exemple bout-en-bout (curl — appels réels)

```
# 1) URL de téléversement — photo de profil recruteur (slot PUBLIC)
curl -s -X POST "$BASE/files/recruteurs/$UUID/profil/upload-url" \
  -H "Authorization: Bearer $JWT" -H "Content-Type: application/json" -d '{"extension":"png"}'

# 2) PUT vers R2 en jouant required_headers (test réel : HTTP 200)
curl -X PUT "<url de l'étape 1>" \
  -H "Content-Type: image/png" -H "Cache-Control: public, max-age=31536000, immutable" \
  --data-binary @photo.png

# 3) Télécharger un PDF d'épreuve PRIVÉ
curl -s "$BASE/files/epreuves/$UUID/file/download-url" -H "Authorization: Bearer $JWT"
# → puis GET l'"url" renvoyée (sans en-tête Authorization) pour récupérer les octets
```

Edukia — module `files` (présignation R2). Généré pour l'intégration mobile. `assets-dev.edukia.net` = hôte public de dev ; l'hôte de prod diffère (le mobile lit toujours l'URL depuis `<slot>_path`).